# GAMEVAULT

Esports Tournament & Team Management System

**COMPREHENSIVE DBMS PROJECT SPECIFICATION & TECHNICAL REPORT**

Complete Architectural Documentation & Demonstration Guide for 20 Distinct MongoDB Functionalities

**Stack: Node.js • Express.js • MongoDB Atlas (Replica Set) • Mongoose ODM**

## 1. Project Abstract & Executive Summary

GameVault is an enterprise-grade backend REST API engineered to organize, orchestrate, and analyze a competitive esports ecosystem. Competitive gaming leagues manage intricate, fast-moving datasets spanning professional players, sponsored multi-game teams, multi-tier tournament brackets, live round scoring, anti-cheat audit trails, and legally binding player contracts.

While traditional relational database management systems (RDBMS) suffer from rigid table schemas and severe performance penalties when resolving deeply nested 5-table JOINs for live esports data, GameVault leverages MongoDB. MongoDB's flexible BSON document model allows intuitive, document-oriented representations of players, rosters, and match progressions while natively supporting multi-document ACID transactions, geospatial indexing, full-text searches, and change streams.

Version 3.0 explicitly implements and demonstrates 20 distinct MongoDB and DBMS functionalities. Each capability is isolated into a dedicated controller, model, and route, and is verified against a live MongoDB Atlas replica set cluster.

## 2. System Architecture & Layered Design

- **Client & Interface Layer:** VS Code REST Client (demo/gamevault.http), cURL, and Postman submitting standard HTTP/REST requests.
- **Routing Layer (Express.js):** Modular Express routers mounted in src/app.js directing incoming requests to specialized controllers.
- **Controller Layer (Business Logic):** Enforces domain logic, query pagination, transaction session management, and change-stream broadcasting.
- **Mongoose ODM Layer:** Provides schema modeling, lifecycle query middleware hooks, virtual population, and index registrations.
- **Database Engine (MongoDB Atlas):** Cloud replica set cluster executing aggregations, $jsonSchema validation, oplog change streams, and GridFS

## 3. Master Feature Map (All 20 Database Capabilities)

Every functionality is fully implemented in the backend, seeded with test data, and verified via HTTP.

| # | FEATURE NAME | CATEGORY | DATABASE CONCEPT | LIVE VERIFIED ENDPOINT |
|---|---|---|---|---|
| 1 | **CRUD Operations** | Baseline | BSON create, read, update, delete | `POST/GET /api/players` |
| 2 | **Relational Linking** | Relational | ObjectId refs & .populate() joins | `GET /api/teams` |
| 3 | **Aggregation Pipeline** | Analytics | $lookup, $addFields, $group | `GET /api/stats/rankings` |
| 4 | **Advanced Indexing** | Optimization | Compound & unique B-tree indexes | `Atlas Indexes Tab` |
| 5 | **Pagination & Sorting** | Optimization | Skip/limit cursor pagination | `GET /api/players?page=1&limit=10` |
| 6 | **Full-Text Search** | Search | Text index & textScore relevance | `GET /api/tournaments/search?q=` |
| 7 | **ACID Transactions** | Integrity | Two-phase commit roster transfer | `POST /api/teams/transfer` |
| 8 | **TTL Auto-Expiration** | Lifecycle | Time-To-Live auto purge (86400s) | `POST /api/invites` |
| 9 | **Geospatial Queries** | Location | GeoJSON Point & 2dsphere $near | `GET /api/tournaments/nearby` |
| 10 | **Change Streams** | Real-Time | Oplog watching & Server-Sent Events | `GET /api/matches/live/stream` |
| 11 | **Faceted Search** | Analytics | $facet parallel pipelines & $bucket | `GET /api/stats/tournaments/faceted` |
| 12 | **Bracket Tree Traversal** | Hierarchy | $graphLookup recursive traversal | `GET /api/tournaments/:id/bracket-tree` |
| 13 | **Capped Event Buffer** | Logging | Fixed-size FIFO circular buffer | `POST/GET /api/audit/events` |
| 14 | **Engine Schema Validation** | Integrity | $jsonSchema database engine rules | `POST /api/contracts (Code 121)` |
| 15 | **Partial Filter Index** | Optimization | RAM-optimized partial index | `GET /api/players/free-agents` |
| 16 | **Positional Array Updates** | Concurrency | Atomic $[elem] via arrayFilters | `PATCH /api/matches/:id/player-stats` |
| 17 | **Soft Delete with Hooks** | Retention | Mongoose pre(/^find/) query hooks | `DELETE /teams/:id/soft, /restore` |
| 18 | **Bulk Write Operations** | Throughput | Batch update via Model.bulkWrite() | `POST /api/matches/bulk-results` |
| 19 | **Linguistic Collation** | Indexing | Case-insensitive index (strength: 2) | `GET /api/players/lookup/tag` |
| 20 | **GridFS Binary Storage** | Storage | Chunked 255KB streaming storage | `POST/GET /api/media` |

## FEATURE 1: CRUD OPERATIONS

BASELINE DATA OPS

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Like managing an inventory in a game: creating player profiles, viewing team rosters, updating rankings, and deleting retired squads.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Fundamental document operations using MongoDB BSON documents. Mongoose maps controllers to insertOne, find, findByIdAndUpdate, and findByIdAndDelete with schema casting.

**Code Files:** src/models/Player.js • src/controllers/playerController.js • src/routes/playerRoutes.js

**Endpoint:** `POST /api/players` • `GET /api/players` • `PUT /api/players/:id` • `DELETE /api/players/:id`

> **Viva Defense:** *"Standard create/read baseline that every advanced query operator builds upon."*

## FEATURE 2: RELATIONAL LINKING & POPULATION

RELATIONAL NOSQL

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Instead of repeating the full player biography inside every team, teams store player IDs and look them up when needed (like a gamer ID badge).

**DBMS & MONGODB ENGINE PRINCIPLE:**

Normalizes relational data using ObjectId references (ref: "Player"). Mongoose .populate() resolves references across collections without document duplication or stale copies.

**Code Files:** src/models/Team.js • src/controllers/teamController.js

**Endpoint:** `GET /api/teams` • `GET /api/teams/:id`

> **Viva Defense:** *"Each team roster holds populated Player objects with username and role, resolving foreign keys."*

## FEATURE 3: AGGREGATION PIPELINE ANALYTICS

DATABASE ANALYTICS

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Calculating team leaderboards and win rates on the fly directly inside the database instead of downloading 10,000 matches into JavaScript loops.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Multi-stage aggregation pipeline ($match, $addFields, $sort, $lookup, $project). Uses $cond and $divide to compute winRate without persisting redundant fields.

**Code Files:** src/controllers/statsController.js • src/routes/statsRoutes.js

**Endpoint:** `GET /api/stats/rankings • GET /api/stats/games-breakdown`

> **Viva Defense:** *"winRate and rosterSize are computed directly on the database server during pipeline execution."*

## FEATURE 4: DATABASE INDEXING (COMPOUND & UNIQUE)

PERFORMANCE OPTIMIZATION

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Like the alphabetical index at the back of a textbook: allows MongoDB to jump directly to a team or tournament in milliseconds instead of scanning every document.

**DBMS & MONGODB ENGINE PRINCIPLE:**

B-tree indexes created via schema.index(). Compound indexes ({ game: 1, status: 1 }) optimize multi-filter queries, while unique indexes prevent handle collisions.

**Code Files:** src/models/Player.js • src/models/Team.js • src/models/Tournament.js

**Endpoint:** `Visualized via MongoDB Atlas Collections > Indexes Tab`

> **Viva Defense:** *"Compound and unique indexes eliminate full collection scans (COLLSCAN) for fast IXSCAN queries."*

## FEATURE 5: PAGINATION & DETERMINISTIC SORTING

LARGE DATASETS

**SIMPLE EXPLANATION & GAMING ANALOGY:**
When viewing leaderboards with 100,000 players, you fetch 10 per page with Next/Previous buttons so your browser never freezes.

**DBMS & MONGODB ENGINE PRINCIPLE:**
Query cursor transformation using .skip((page - 1) * limit) and .limit(limit). Accompanied by countDocuments to return total pages for client pagination.

**Code Files:** src/utils/pagination.js • src/controllers/playerController.js
**Endpoint:** `GET /api/players?page=1&limit=10&sort=username`

> **Viva Defense:** *"Every list endpoint accepts page, limit, and sort query parameters to handle massive datasets."*

## FEATURE 6: FULL-TEXT SEARCH & RELEVANCE SCORING

SEARCH ENGINE

**SIMPLE EXPLANATION & GAMING ANALOGY:**
A search bar that lets users type keywords like "invitational" or "qualifier" to find tournaments matching titles, descriptions, or rulebooks.

**DBMS & MONGODB ENGINE PRINCIPLE:**
Compound text index on { name: "text", description: "text", rules: "text" }. Queries use $text with $search and sort by textScore relevance metadata.

**Code Files:** src/models/Tournament.js • src/controllers/tournamentController.js
**Endpoint:** `GET /api/tournaments/search?q=invitational`

> **Viva Defense:** *"MongoDB tokenizes words, stems terms, and scores results based on term frequency and relevance."*

## FEATURE 7: MULTI-DOCUMENT ACID TRANSACTIONS

DATA CONSISTENCY

**SIMPLE EXPLANATION & GAMING ANALOGY:**

An all-or-nothing transfer guarantee: when a player moves from Team A to Team B, both rosters and the player document update together. If power fails mid-way, it rolls back.

**DBMS & MONGODB ENGINE PRINCIPLE:**

session.withTransaction() on a replica set. Coordinates two-phase commit across multiple documents; if any write fails, all operations are cleanly rolled back.

**Code Files:** src/controllers/teamController.js (transferPlayer)

**Endpoint:** `POST /api/teams/transfer`

> **Viva Defense:** *"Guarantees atomic roster moves so a player can never be trapped on two rosters or left without one."*

## FEATURE 8: TIME-TO-LIVE (TTL) AUTOMATIC EXPIRATION

LIFECYCLE AUTOMATION

**SIMPLE EXPLANATION & GAMING ANALOGY:**

A self-destruct countdown timer on scrim invites: pending invitations automatically vanish 24 hours after creation without running background cron jobs.

**DBMS & MONGODB ENGINE PRINCIPLE:**

TTL single-field index on createdAt with expires: 86400 (24 hours). MongoDB background thread monitors index timestamps and purges expired documents automatically.

**Code Files:** src/models/MatchInvite.js • src/controllers/inviteController.js

**Endpoint:** `POST /api/invites • GET /api/invites`

> **Viva Defense:** *"Documents are automatically erased by MongoDB storage engine when TTL timer expires."*

## FEATURE 9: GEOSPATIAL QUERIES (2DSPHERE & $NEAR)

LOCATION SERVICES

**SIMPLE EXPLANATION & GAMING ANALOGY:**

A LAN tournament locator: players enter their city coordinates and the API finds in-person gaming arenas within a 100km radius sorted by proximity.

**DBMS & MONGODB ENGINE PRINCIPLE:**

GeoJSON Point storage ({ type: "Point", coordinates: [lng, lat] }) paired with a 2dsphere index. Queried using $near and $maxDistance in spherical meters.

**Code Files:** src/models/Tournament.js • src/controllers/tournamentController.js

**Endpoint:** `GET /api/tournaments/nearby?lng=-122.42&lat=37.77&maxDistance=100000`

> **Viva Defense:** *"Calculates geodesic distance over the Earth sphere and automatically sorts results nearest-first."*

## FEATURE 10: REAL-TIME CHANGE STREAMS & SERVER-SENT EVENTS

REAL-TIME PUSH

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Live spectator scoreboard: as soon as a referee updates the match score, all connected spectators see the score update immediately without refreshing.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Match.watch() opens a persistent cursor against the MongoDB replica set internal oplog. Changes trigger Node.js EventEmitter pushing data over Server-Sent Events (SSE).

**Code Files:** src/utils/changeStreamWatcher.js • src/controllers/matchController.js

**Endpoint:** `GET /api/matches/live/stream (SSE) • PATCH /api/matches/:id/score`

> **Viva Defense:** *"Watches the database oplog directly to stream real-time push events to connected viewers."*

## FEATURE 11: FACETED SEARCH & MULTI-BUCKET ANALYTICS

MULTI-DIMENSIONAL STATS

**SIMPLE EXPLANATION & GAMING ANALOGY:**
The filter sidebar on Steam: clicking one search returns tournament prize brackets, game categories, and top tournaments calculated simultaneously.

**DBMS & MONGODB ENGINE PRINCIPLE:**
The $facet aggregation stage executes multiple independent pipelines in parallel over input documents in a single round-trip, utilizing $bucket and $sortByCount.

**Code Files:** src/controllers/statsController.js (getFacetedTournamentStats)
**Endpoint:** `GET /api/stats/tournaments/faceted`

> **Viva Defense:** *"Computes prize pool buckets, game breakdowns, and top tournaments in a single database trip."*

## FEATURE 12: RECURSIVE GRAPH TRAVERSAL ($GRAPHLOOKUP)

HIERARCHICAL GRAPHS

**SIMPLE EXPLANATION & GAMING ANALOGY:**
Esports bracket tree: Quarterfinals feed into Semifinals, which feed into Finals. MongoDB traces the whole tournament tree backwards in one call.

**DBMS & MONGODB ENGINE PRINCIPLE:**
$graphLookup performs recursive transitive closure search across documents, linking previousMatchIds to _id and tagging recursion depth (bracketLevel).

**Code Files:** src/models/Match.js • src/controllers/tournamentController.js
**Endpoint:** `GET /api/tournaments/:id/bracket-tree`

> **Viva Defense:** *"Traverses directed acyclic graph (DAG) hierarchies natively without complex recursive app code."*

## FEATURE 13: CAPPED COLLECTIONS & CIRCULAR FIFO BUFFER

HIGH-THROUGHPUT TICKER

**SIMPLE EXPLANATION & GAMING ANALOGY:**

An in-game killfeed ticker: you only care about the latest 500 events. Old events fall off the back of the conveyor belt automatically as new ones arrive.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Fixed-size collection created with capped: true, size: 1MB, max: 500. Inserts preserve natural physical order on disk, retrieved via $natural: -1 with zero cleanup jobs.

**Code Files:** src/models/MatchEventLog.js • src/controllers/auditController.js

**Endpoint:** `POST /api/audit/events • GET /api/audit/events?limit=10`

> **Viva Defense:** *"Maintains strict FIFO order on disk and auto-purges oldest records once byte/count limit is reached."*

## FEATURE 14: ENGINE-LEVEL SCHEMA VALIDATION ($JSONSCHEMA)

DATABASE GUARDRAILS

**SIMPLE EXPLANATION & GAMING ANALOGY:**

An unbreakable rulebook inside MongoDB: player contracts must have salary >= $500 and a code like CTR-####-XX. Invalid writes are blocked even in Compass.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Native MongoDB collection validator using $jsonSchema. Rejects invalid documents at the storage engine level with error code 121 (Document failed validation).

**Code Files:** src/models/PlayerContract.js • src/controllers/contractController.js

**Endpoint:** `POST /api/contracts (valid: 201 Created; invalid salary/code: 400 with code 121)`

> **Viva Defense:** *"Proves database-level constraint enforcement independent of application-layer Mongoose validation."*

## FEATURE 15: PARTIAL FILTER INDEXES (PARTIALFILTEREXPRESSION)

INDEX OPTIMIZATION

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Scouts only look for Free Agents (players without a team). Indexing all 50,000 rostered players is a waste of RAM, so we only index players with team == null.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Index defined with partialFilterExpression: { team: null }. Ignores all documents not matching the predicate, drastically reducing index memory and write overhead.

**Code Files:** src/models/Player.js • src/controllers/playerController.js

**Endpoint:** `GET /api/players/free-agents`

> **Viva Defense:** *"Significantly reduces B-tree index footprint by only indexing relevant subset data."*

## FEATURE 16: POSITIONAL ARRAY UPDATES (ARRAYFILTERS & $INC)

ATOMIC SUB-DOCUMENTS

**SIMPLE EXPLANATION & GAMING ANALOGY:**

A player gets a kill during a match: the database reaches straight inside the match player list and increments their kills by +1 atomically without race conditions.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Positional array operator $[p] paired with arrayFilters: [{ "p.player": playerId }] and $inc. Updates the specific array sub-document without full document replacement.

**Code Files:** src/models/Match.js • src/controllers/matchController.js

**Endpoint:** `PATCH /api/matches/:id/player-stats`

> **Viva Defense:** *"Atomic in-place embedded array mutation avoiding read-modify-write race conditions in live scoring."*

## FEATURE 17: SOFT DELETE WITH MONGOOSE LIFECYCLE HOOKS

DATA RETENTION & INTEGRITY

**SIMPLE EXPLANATION & GAMING ANALOGY:**

A recycle bin for disbanded teams: marking isDeleted: true keeps past match history intact while automatically hiding the team from normal app searches.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Mongoose pre(/^find/) and pre("countDocuments") middleware hooks intercept queries and inject { isDeleted: { $ne: true } } unless includeDeleted: true is passed.

**Code Files:** src/models/Team.js • src/controllers/teamController.js

**Endpoint:** `DELETE /api/teams/:id/soft • PATCH /api/teams/:id/restore • GET /api/teams?includeDeleted=true`

> **Viva Defense:** *"Demonstrates query lifecycle middleware and transparent enterprise data retention."*

## FEATURE 18: HIGH-THROUGHPUT BULK WRITE OPERATIONS

BATCH THROUGHPUT

**SIMPLE EXPLANATION & GAMING ANALOGY:**

Submitting 20 match scores at the end of a tournament day in one single package instead of making 20 slow back-and-forth network trips.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Model.bulkWrite() batches multiple write operations (updateOne, insertOne) into a single network protocol packet, minimizing round-trips and database lock contention.

**Code Files:** src/controllers/matchController.js (bulkUpdateMatches)

**Endpoint:** `POST /api/matches/bulk-results`

> **Viva Defense:** *"Bundles dozens of match outcome writes into a single high-throughput database roundtrip."*

## FEATURE 19: LINGUISTIC COLLATION (CASE-INSENSITIVE INDEXING)

STRING MATCHING

**SIMPLE EXPLANATION & GAMING ANALOGY:**

In esports, "v1per", "V1PER", and "V1per" are the same person. The database understands letter-case rules so duplicate handles are prevented and lookups are fast.

**DBMS & MONGODB ENGINE PRINCIPLE:**

Index collation { locale: "en", strength: 2 }. Level 2 strength ignores case differences during comparison and sorting, eliminating slow regex table scans.

**Code Files:** src/models/Player.js • src/controllers/playerController.js

**Endpoint:** `GET /api/players/lookup/tag?handle=V1PER`

> **Viva Defense:** *"Enforces case-insensitive handle uniqueness and search directly at the B-tree index level."*

## FEATURE 20: GRIDFS CHUNKED BINARY MEDIA STORAGE

LARGE FILE STORAGE

**SIMPLE EXPLANATION & GAMING ANALOGY:**

MongoDB documents are capped at 16MB. GridFS splits official 50MB PDF rulebooks and match replay demo files into small 255KB chunks to store and stream them.

**DBMS & MONGODB ENGINE PRINCIPLE:**

GridFSBucket divides large binary files into 255KB chunks stored in media.files and media.chunks collections, streaming bytes cleanly to clients with MIME headers.

**Code Files:** src/controllers/mediaController.js • src/routes/mediaRoutes.js

**Endpoint:** `POST /api/media/upload • GET /api/media • GET /api/media/:id/stream`

> **Viva Defense:** *"Overcomes the 16MB BSON limit by chunking and streaming binary files via MongoDB GridFS."*

## 4. Key Academic & Viva Defense Questions

Fundamental theoretical rationales and architecture defense points for professor evaluation.

**Q1: Why choose MongoDB over a Relational Database (SQL) for GameVault?**

Esports data is inherently hierarchical and dynamic (nested team rosters, multi-tier tournament brackets, real-time match telemetry). In SQL, querying a tournament with its participating teams, player rosters, and live matches requires complex 5-table JOIN operations that introduce substantial lock contention. MongoDB stores documents in naturally aligned structures while still supporting multi-document ACID transactions (Feature 7) where relational integrity is critical.

**Q2: What is the exact difference between Mongoose Validation and MongoDB $jsonSchema?**

Mongoose validation runs strictly in the Node.js application process memory before data is transmitted. If an administrator connects via MongoDB Compass, Python, or shell, Mongoose validation is completely bypassed. In contrast, Feature 14 enforces $jsonSchema directly inside the MongoDB database storage engine, rejecting invalid writes with error code 121 regardless of the client used.

**Q3: Why use a Capped Collection for Match Events instead of a TTL Index?**

TTL indexes delete records based on elapsed duration (e.g., after 24 hours). Capped Collections (Feature 13) enforce strict physical size (1MB) and document count (500) boundaries. When record #501 arrives, record #1 is overwritten immediately on disk like a circular buffer with zero background delete overhead, guaranteeing fixed disk allocation for high-throughput live event tickers.

**Q4: How do Change Streams function internally?**

Change Streams (Feature 10) require a MongoDB Replica Set. Every state modification is recorded into the internal replication operations log (oplog.rs). Match.watch() establishes a real-time tailable cursor against this oplog, emitting instant change events that are streamed over Server-Sent Events (SSE) without client polling.

**Q5: How does GridFS handle high-volume binary file downloads?**

GridFS divides files exceeding 16MB into 255KB chunks. When a user requests a tournament rulebook or demo replay, the GridFSBucket opens a download stream that fetches chunks sequentially and pipes binary buffers directly to the Express response socket with appropriate MIME headers, minimizing server memory pressure.

## Team Details

**Nikit Pandey(24BKT0165) , Aarav Gupta (24BCE0725) , Raunak Raja (24BCE0057)**